

1. Performance Graphs

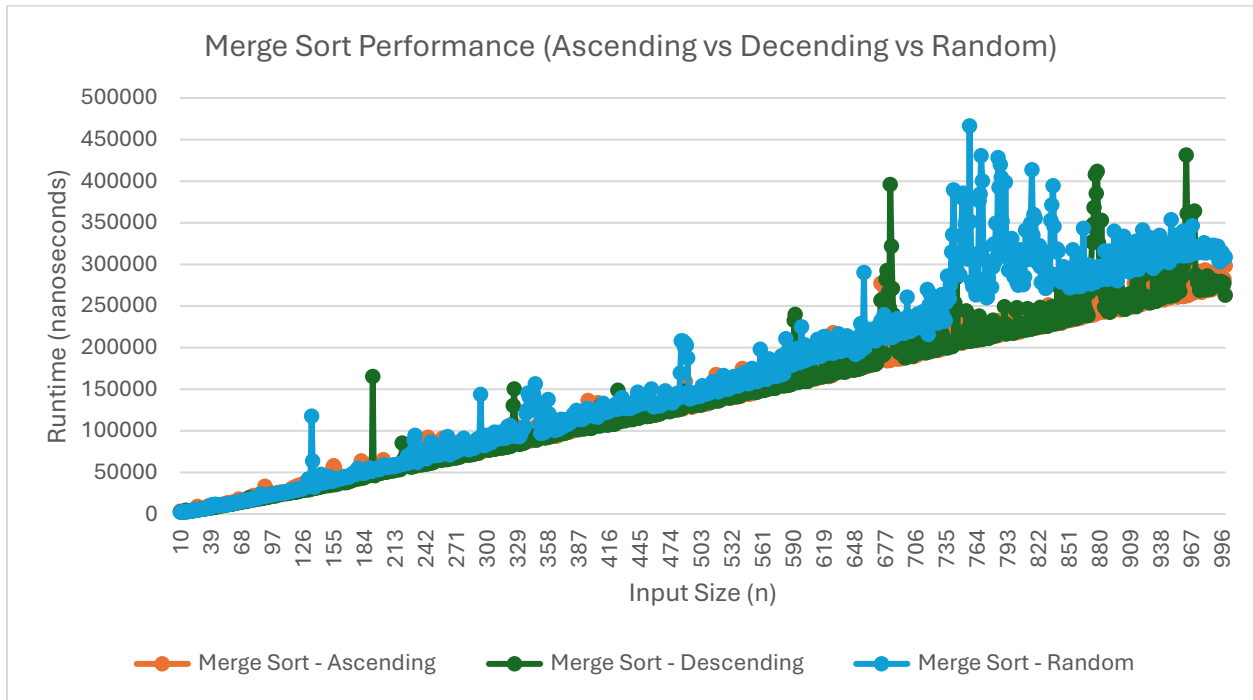


Figure 1. Merge Sort Performance (Ascending vs Descending vs Random)

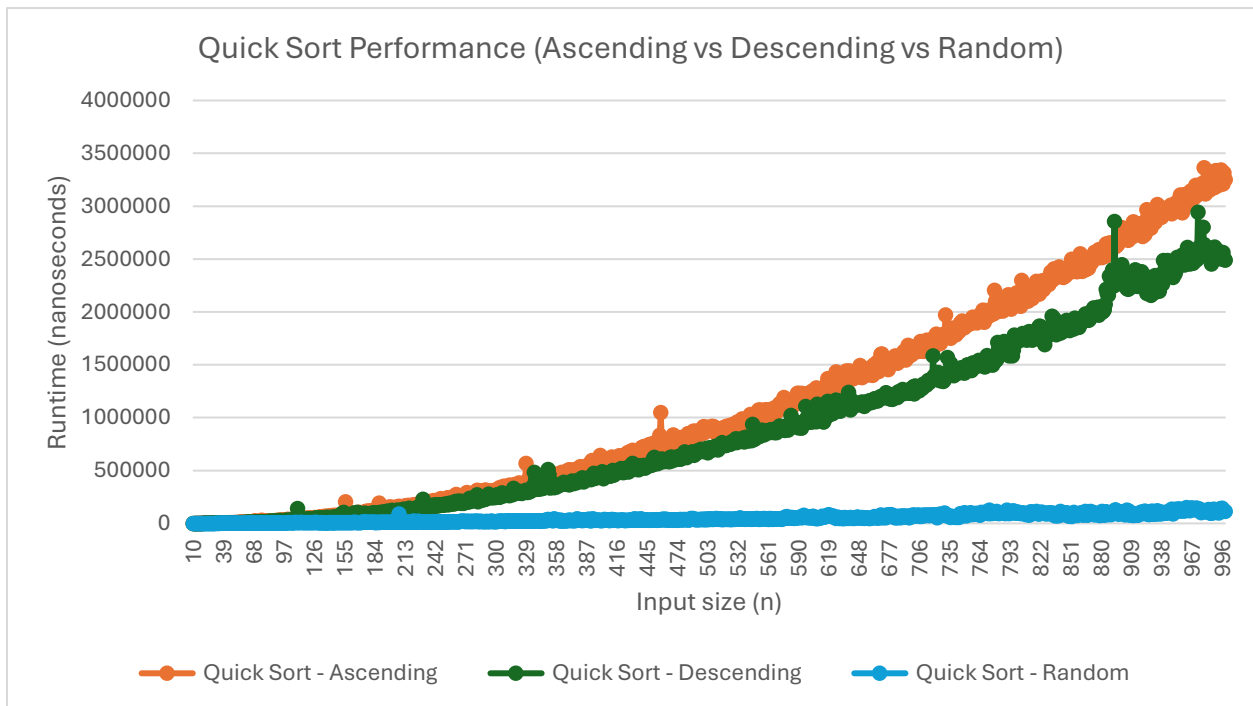


Figure 2. Quick Sort Performance (Ascending vs Descending vs Random)

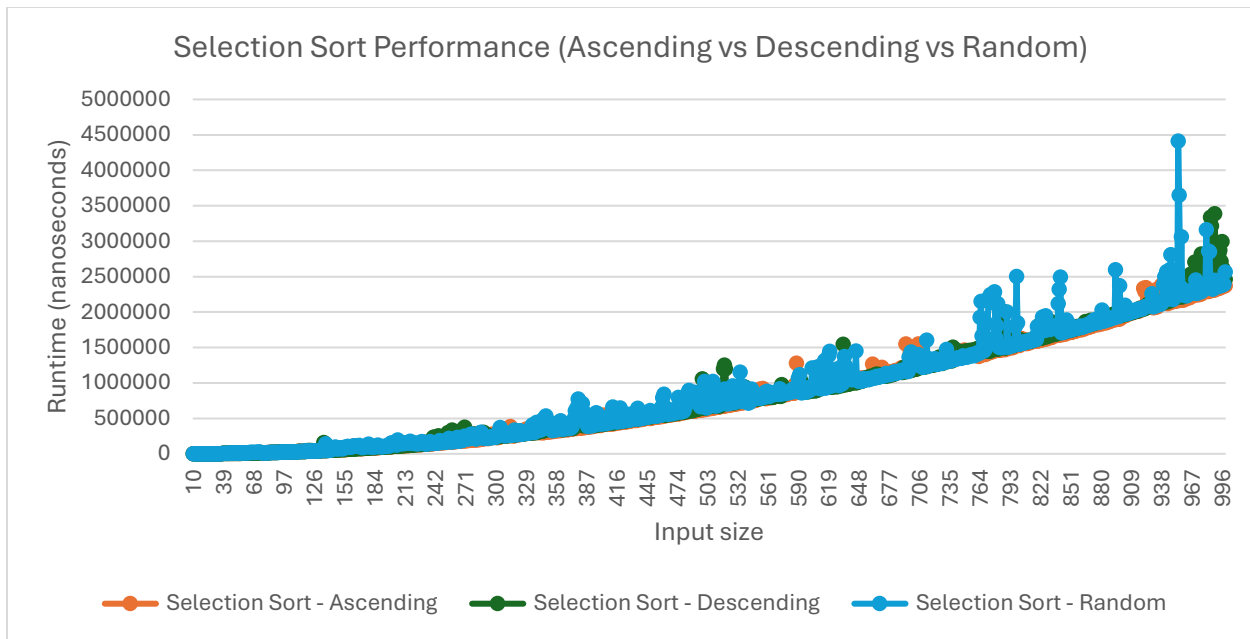


Figure 3. Selection Sort Performance (Ascending vs Descending vs Random)

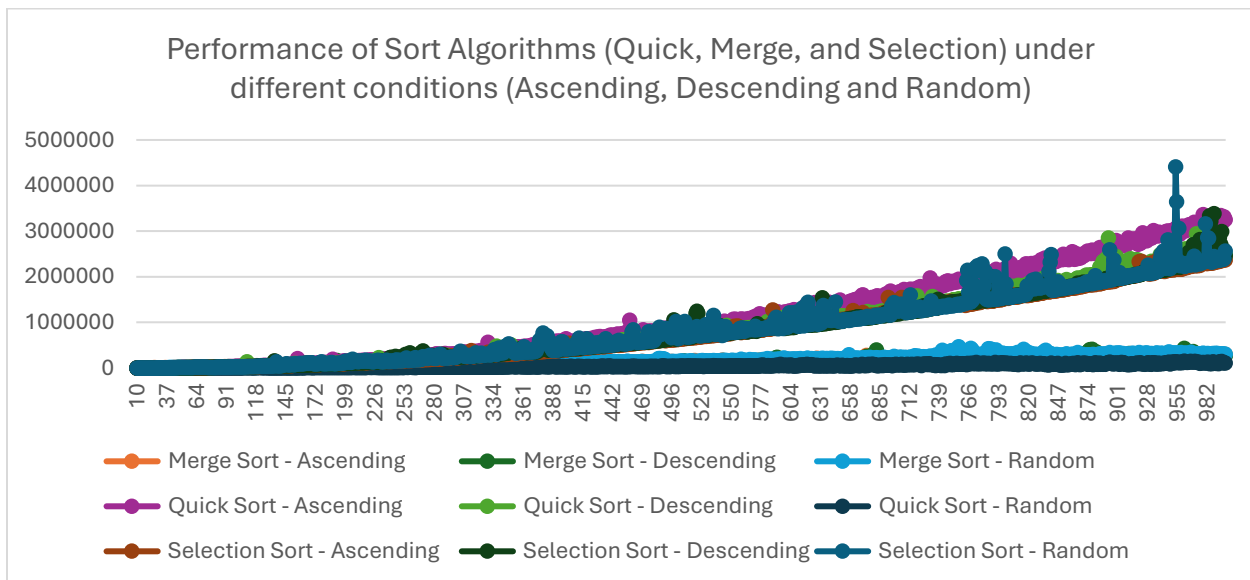


Figure 4. Performance of Sort Algorithms (Quick, Merge, and Selection) under different conditions (Ascending, Descending and Random)

2. What's Going On?

Q1. Why do Quick Sort and Selection Sort behave differently depending on initial configuration, while Merge Sort remains consistent?

Merge Sort's performance is consistent because its algorithmic structure always performs the same sequence of operations regardless of the initial ordering of the data. Merge sort repeatedly divides the list in half and merges sorted sub lists. Because it always performs the same division pattern and the merging step always processes all elements, Merge Sort always runs in $\Theta(n \log n)$ time. This explains why, in all three of your Merge Sort graphs—Ascending, Descending, and Random—the data points follow nearly the same linear-log growth pattern with only small fluctuations.

Quick Sort, however, is highly sensitive to the arrangement of the input because of how the pivot is chosen. The implementation used in this assignment follows **Lomuto partitioning**, which always chooses the **last element as the pivot**. Because of this:

- **Ascending input $\rightarrow$ worst case ($O(n^2)$)**, since every pivot becomes the largest element.
- **Descending input $\rightarrow$ also near-worst case**, since every pivot becomes the smallest element.
- **Random input $\rightarrow$ expected case $\Theta(n \log n)$** , because pivots are likely to split the array more evenly.

This explains why in your Quick Sort graph, the Ascending and Descending lines grow extremely steeply, reaching several million nanoseconds, while the Random case stays much lower and closer to the theoretical average-case performance.

Selection Sort also shows differences between the cases, although not as dramatically. The algorithm always performs n^2 **comparisons**, but its number of swaps differs per configuration. When the array is already sorted in ascending order, Selection Sort performs the minimal number of swaps ($n - 1$). When the data is descending, or otherwise highly disordered, it performs significantly more swaps. Because swapping is more expensive than comparing, the Descending and Random cases show slightly worse performance, even though Selection Sort theoretically remains $\Theta(n^2)$ in all cases.

In summary:

- **Merge Sort:** behavior independent of input
- **Quick Sort:** highly dependent on input due to pivot choice
- **Selection Sort:** still $\Theta(n^2)$, but swap count creates visible differences

This is exactly what the theory predicts, and your graphs clearly visualize these effects.

Q2. Under what scenario would you choose each sorting algorithm based solely on runtime?

Based strictly on performance shown in Graph #4, we can make the following decisions:

Merge Sort — Choose for almost all real-world data

Merge Sort consistently outperforms the worst cases of Quick Sort and Selection Sort. It is predictable, stable, and always $\Theta(n \log n)$.

You would choose Merge Sort when:

- Data may be sorted, reverse-sorted, or random
- Predictable performance is required
- Worst-case speed matters (e.g., large datasets, time-sensitive workloads)

Quick Sort — Choose only when data is random or well-shuffled

Quick Sort on random input is very fast and often outperforms Merge Sort in practice due to good cache locality.

You would choose Quick Sort when:

- Input data is random or has been intentionally randomized
- You need fast average-case performance
- Memory usage must be minimized (in-place algorithm)

However, **you should never choose this Quick Sort implementation for already-sorted or reverse-sorted data**, since it hits its worst case.

Selection Sort — Almost never choose

Selection Sort performs the worst in your combined graph, and its $\Theta(n^2)$ scaling makes it unusable for medium or large datasets.

You might choose Selection Sort only when:

- Input size is extremely small ($n < 20$)
- Memory writes must be minimized (good for embedded systems)
- Implementation simplicity matters more than speed

Otherwise, Merge Sort or Quick Sort are superior choices.

3. Did We Learn Anything?

Choosing a sorting algorithm requires more than simply comparing theoretical runtimes. Real-world software systems operate under constraints such as memory limitations, worst-case guarantees, implementation complexity, reliability requirements, and maintainability. While this lab highlights runtime behavior through empirical measurement, understanding these additional tradeoffs is essential for practical algorithm selection.

One major consideration is **space overhead**. Merge Sort requires additional memory proportional to the size of the input ($O(n)$ auxiliary space). This makes Merge Sort less appealing in environments where memory is limited, such as embedded devices or systems with strict memory budgets. On the other hand, Quick Sort and Selection Sort are in-place algorithms, meaning they use constant auxiliary space ($O(1)$). For memory-constrained environments, the in-place nature of Quick Sort makes it attractive—provided the worst-case runtime can be avoided or mitigated.

Another factor is **implementation complexity**. Selection Sort is extremely simple to implement and maintain but performs poorly for large datasets. Merge Sort is more complex, involving recursive division and merging, and Quick Sort's partitioning logic is notoriously error prone. In industrial contexts, simpler code often reduces bugs and improves maintainability. That said, many languages provide library implementations (e.g., C++ `std::sort`, Python's Tim sort) that abstract away the complexity and offer hybrid or optimized algorithms.

Worst-case performance guarantees are also critical. Merge Sort guarantees $\Theta(n \log n)$ time no matter the input configuration. This makes it well-suited for systems where performance predictability is essential, such as real-time transaction systems, databases, and latency-sensitive services. Quick Sort, however, suffers from a worst-case $\Theta(n^2)$ runtime unless pivot selection is randomized or optimized using strategies like median-of-three. Selection Sort reliably runs in $\Theta(n^2)$, but that predictability is overshadowed by its consistently poor performance.

A further consideration is **stability**—whether equal values maintain their original relative order. Merge Sort is stable by nature, making it useful for multi-key sorting. Quick Sort and Selection Sort are not stable without modification. When building user-facing software or data processing systems where stable ordering matters, Merge Sort becomes the preferred choice.

Finally, **real-world performance does not always match theoretical expectations**, as seen in your graphs. Quick Sort is theoretically faster than Merge Sort on average, yet your measurements show that this depends heavily on pivot strategy and input arrangement.

The lab effectively illustrates that algorithmic analysis provides a high-level understanding, but empirical testing is essential to understand behavior on specific inputs and implementations.

In conclusion, choosing a sorting algorithm depends on a balance of runtime, memory usage, input characteristics, stability, and code complexity. Merge Sort offers consistent performance and stability, Quick Sort offers excellent average performance under suitable conditions, and Selection Sort offers simplicity but weak runtime scalability. Effective algorithm selection requires evaluating all these factors rather than relying solely on Big-O notation.